AppGuard AI

Understanding Android app permissions with explainable machine learning.

The project in one sentence

AppGuard AI learns from examples of benign and malware-labeled Android apps, then estimates whether a new permission pattern looks more like one group or the other.

What is a permission?

A permission is access an app requests, such as permission to use the camera, read text messages or connect to the internet. A permission can have a legitimate purpose. Requesting it does not automatically make an app malicious.

What does the user do?

1. Enter permissions. Search and select permissions, or paste a list.

2. Run the model. The app calculates a score using a trained machine-learning model.

3. Read the explanation. See which present or absent permissions pushed the model score up or down.

4. Explore and export. Try a different permission set and download an analysis report.

An easy way to think about it

Imagine a student learning from many labeled examples. After learning, the student receives a new example and makes an educated guess. The model does something similar with permission patterns. It can still make mistakes.

Important: This is a research and learning project, not an antivirus. It does not scan an APK, inspect app code or prove that an app is safe.

This guide explains the implementation already built in this project. The dataset was created by your instructor, as you reported.

How the project works

The real dataset

What the file contains	Actual count
App records	11,274
Permission inputs	195
Benign / malware labels	5,721 / 5,553
Distinct permission patterns	3,273

The Excel file has 197 columns in total: 195 permissions, one label and one app ID. The label and ID are not given to the model as input.

From permissions to a prediction

Encode. Each permission becomes a number: 1 means selected and 0 means absent. Every prediction uses the same column order as training.

Learn. Compare logistic regression, Random Forest and Extra Trees. A simple class-frequency baseline provides a reference.

Choose. Use validation F1 to pick the model. Extra Trees won for this experiment. It combines many randomized decision trees to produce a score.

Test. Evaluate the chosen model on examples held back from model selection.

Why the split is an important feature

Many apps share exactly the same permission pattern. Putting matching patterns in both training and test would be like showing a student some of the exam questions while teaching. This project keeps identical patterns together in one split.

Training: 6,765 apps for initial learning. **Validation:** 2,253 apps for model choice. **Test:** 2,256 apps for final evaluation. After selection, the chosen model learns again from training plus validation; the test set remains held out.

There are 85 patterns with both benign and malware labels. This shows why permissions alone cannot fully determine whether an app is malicious. Grouping reduces repeated-pattern leakage, but does not prove generalization to future malware.

What the results mean

These are measured results for the selected Extra Trees model on the separate test set. Malware is the positive class.

Metric	Result	In simple words
Accuracy	89.63%	How often the predicted label was correct overall.
Precision	96.01%	Of apps flagged as malware-like, how many had a malware label?
Recall	82.36%	Of malware-labeled apps, how many did the model detect?
F1 score	88.66%	A score balancing precision and recall.
ROC-AUC	0.9704	How well scores rank malware above benign apps across thresholds.

The actual test outcomes

	Predicted benign-like	Predicted malware-like
Actually benign	1,107 correct	38 false alarms
Actually malware	196 missed	915 detected

The limitation to remember: The model missed 196 malware-labeled apps. A high precision score does not mean it detects every threat.

A probability is an estimate, not a guarantee

A score of 80% means the model assigned 0.80 to the malware class. It does not establish an 80% real-world chance that the app is malware. The model has not undergone a separate probability-calibration step.

Classification uses a 50% threshold. The display bands are Low below 30%, Moderate from 30% to below 70%, and High from 70%. These bands are illustrative, not independently validated security rules.

These results are from this dataset and this split. They do not reproduce the earlier 95.15% research claim or demonstrate production security performance.

How to explain your work

What is SHAP?

SHAP explains how the model arrived at a particular score. It starts from a reference score and assigns contributions to the permissions. For the selected model, the reference plus all contributions adds up to the predicted probability.

Coral bars increase the malware score. Teal bars decrease it. A missing permission can also influence a prediction. These bars explain learned associations; they do not prove that a permission causes malware.

The tools and their jobs

Tool	Job in this project
Python and pandas	Read, validate and prepare the data.
scikit-learn	Train and evaluate the classifiers.
SHAP	Explain individual predictions.
Streamlit and Plotly	Provide the interactive app and charts.
pytest	Check behavior automatically; 24 tests passed locally.

A short interview introduction

"I built an explainable Android permission-classification app using an instructor-created dataset. I compared three classifiers and kept matching permission patterns together to reduce leakage between splits. Extra Trees achieved 88.66% test F1. The app includes real predictions, SHAP explanations, what-if comparisons and an evidence dashboard."

A three-minute demo

Start. Launch the app and show the overview. Select INTERNET, READ_SMS and SEND_SMS as example inputs, not as a known malicious app.

Explain. Run analysis, read the actual score and show the SHAP chart. Export the HTML report.

Evaluate. Show the test results, especially missed malware. Explain why a lower what-if score does not prove that an app became safer.

Credit your instructor for the dataset and be candid about AI-assisted development. Learn the code before claiming implementation skills. A useful next step is evaluation on an independently sourced, newer dataset.

Prepare your repository

A GitHub repository is an online project folder with a history of changes. Uploading the code makes it reviewable; it does not automatically host a running Streamlit app.

1. Extract the prepared ZIP. Use dist/appguard-ai-source.zip. Right-click it in File Explorer and choose Extract All. Open the inner appguard-ai folder.

2. Create an empty repository. Sign in to GitHub and open https://github.com/new. Name it appguard-ai. Choose Public for a recruiter-visible portfolio, or Private while preparing.

3. Add a short description. Use: "Explainable Android permission classification with machine learning, SHAP and Streamlit." Leave automatic README, .gitignore and license creation off for this initial upload; the project already includes the first two.

4. Upload the extracted contents. Click Create repository, then uploading an existing file. In an existing repo, use Add file > Upload files. Drag the files and folders INSIDE appguard-ai onto the upload page. Do not upload only the ZIP or the outer folder.

5. Include configuration folders. In File Explorer enable View > Show > Hidden items if needed. Check that .github, .streamlit and .gitignore are included along with app.py, README.md, appguard, tests and assets.

6. Commit the upload. Use the message "Initial upload: AppGuard AI" and click Commit changes. A commit is a saved version of the project.

7. Check the result. app.py and README.md should appear at the repository root. Check that the README image loads. Open Actions to review tests; model-dependent tests intentionally skip in the source-only copy.

8. Add the guide and share. Optionally upload this PDF inside a docs folder. Add topics such as machine-learning, explainable-ai, android and streamlit. Share the repository URL on your resume.

What is intentionally excluded?

The source ZIP excludes Combined.xlsx, the trained model, row-level evaluation files and the virtual environment. Obtain instructor permission before sharing data or derived model artifacts. A public clone needs authorized model artifacts or training data to run predictions. Your original local project already has them.

No GitHub repository or public deployment was created as part of this guide. Do not use the localhost address as a public demo link.

Project evidence: README.md, train_model.py and models/metrics.json in the local project. GitHub instructions checked against official documentation on 10 November 2026:

GitHub: Creating a new repository
GitHub: Adding a file to a repository